
Playwright with TypeScript — Top 75 Real-Time Interview Questions & Answers

Advanced / Scenario-Based Q&A for Experienced SDET & QA Automation Engineers

75 curated, scenario-based questions covering locators, waiting strategies, test architecture, fixtures, API mocking, authentication, CI/CD, debugging, visual testing, and TypeScript-specific best practices — for engineers preparing for mid-to-senior SDET / QA Automation interviews.

Table of Contents

1. Locators, Selectors & Element Handling

1. Why does Playwright recommend user-facing locators (getByRole, getByLabel, getByText) over CSS/XPath in real projects?
2. What is the difference between locator.first(), locator.nth(), and locator.last(), and when would using them be a code smell?
3. How do you scope a locator to search only within a specific parent element (real-world table/row scenario)?
4. What is the difference between locator.filter() and passing a hasText/has option directly in locator()?
5. How do you handle a dynamic list where you need to assert the count of elements before interacting with any of them?
6. How do you build a robust locator strategy when developers don't add data-testid attributes and the design changes often?
7. How do you interact with elements inside an iframe using Playwright?
8. How do you interact with elements inside a Shadow DOM in Playwright?
9. How would you handle a custom dropdown/autocomplete component that isn't a native <select> element?
10. How do you handle native <select> dropdowns versus custom-rendered ones?

2. Waiting Strategies & Test Reliability

11. Explain Playwright's auto-waiting mechanism and which conditions it checks before an action like click().
12. If auto-waiting exists, why do flaky tests still happen, and how do you fix them in real projects?

13. What is the difference between `page.waitForSelector()` and `expect(locator).toBeVisible()`?
14. How does `expect.poll()` differ from a normal web-first assertion, and when would you use it in a real scenario?
15. How do you avoid hard-coded waits (`page.waitForTimeout()`) in a CI pipeline where tests need to be reliable and fast?
16. How do you handle a page where content loads via multiple sequential API calls (e.g., a dashboard that fires several XHRs on load)?
17. What causes a "strict mode violation" error in Playwright and how do you resolve it?
18. How do you handle elements that are visible in the DOM but covered by another element (e.g., a sticky header or a cookie banner)?

3. Test Architecture, Fixtures & Page Object Model

19. How do you implement the Page Object Model (POM) in Playwright using TypeScript in a maintainable way?
20. How do custom fixtures in Playwright Test improve on a traditional POM-only setup?
21. What is the difference between test-scoped and worker-scoped fixtures, and give a real use case for worker scope?
22. How do you merge fixtures from multiple files/plugins (e.g., combining your own POM fixtures with an auth fixture)?
23. How do you structure a large Playwright project (100+ specs) so it stays maintainable across multiple teams?
24. What is the "test.step" API and why is it valuable for real-world debugging and reporting?
25. How do you avoid test interdependence (tests relying on order/state from a previous test) in Playwright?

4. API Testing, Network Interception & Mocking

26. How do you make a pure API request (no browser) using Playwright, and why would you do that in an E2E suite?
27. How do you intercept and mock a network response to test a UI's error-handling state (e.g., a 500 error banner)?
28. What is the difference between `route.fulfill()`, `route.continue()`, and `route.abort()`?
29. How would you test a page that depends on a slow third-party API without making your suite slow or flaky?
30. How do you validate the actual request payload sent by the frontend (e.g., asserting a POST body is correct) rather than just the response?
31. How do you test a page that uses WebSockets, and can Playwright intercept WebSocket traffic?
32. How do you globally mock all API calls for an entire test file or test run (not just one test)?

5. Authentication, Sessions & Cross-Cutting Setup

33. How do you avoid logging in through the UI in every single test, which is slow and adds unrelated flakiness?
34. How do you configure `playwright.config.ts` to run an auth "setup" project before the actual test projects, and reuse its output?
35. How do you handle multiple user roles (admin, standard user, read-only) that each need separate authenticated sessions?
36. How do you handle authentication for an app protected by SSO/OAuth (e.g., Google or Okta login) in automated tests?
37. How do you handle a token that expires mid-suite (e.g., a JWT with a short TTL) so long test runs don't start failing due to auth expiry?
38. How do you seed test data (e.g., a specific user, a specific product) before a test and clean it up afterward, in a way that keeps tests independent?

6. Parallelization, CI/CD & Execution Strategy

- 39. How does Playwright Test achieve parallelism, and what's the difference between "workers" and "projects"?
- 40. What is the difference between `fullyParallel: true` and the default parallel mode?
- 41. How do you implement test sharding across multiple CI machines, and how is it different from workers?
- 42. How do you merge HTML/JSON reports produced by multiple shards into a single report?
- 43. What retry strategy would you configure for CI vs local development, and why should retries differ?
- 44. How do you set up Playwright to run inside Docker for CI, and what's a common pitfall?
- 45. How do you run only a subset of tests (e.g., "smoke" tests) in a fast pre-merge CI check, while running the full suite nightly?

7. Debugging, Trace Viewer & Observability

- 46. What is the Playwright Trace Viewer and what real debugging value does it add over screenshots/logs alone?
- 47. What's the tradeoff of setting trace: 'on' for every run versus 'on-first-retry' or 'retain-on-failure'?
- 48. How do you debug a single failing test locally with step-by-step control (not just headless output)?
- 49. What is Playwright's UI Mode and how does it help in day-to-day development?
- 50. How do you capture and attach custom debugging artifacts (e.g., a database dump or a specific API response) to the Playwright HTML report for a failing test?
- 51. A test passes locally but consistently fails in CI. What is your systematic approach to diagnosing this?

8. Assertions, Visual Testing & Cross-Browser

- 52. What is a "web-first assertion" in Playwright and how does it differ from a plain Node.js `assert/expect` equality check?
- 53. What is a "soft assertion" and when is it appropriate to use in a real test?
- 54. How do you implement visual regression testing with Playwright, and what's the biggest real-world challenge with it?
- 55. How do you handle animations/transitions that cause visual tests to be flaky?
- 56. How do you run the same test suite across Chromium, Firefox, and WebKit, and what real cross-browser issues commonly surface?
- 57. How do you test responsive/mobile layouts without a real device farm?

9. Advanced Real-World Scenarios

- 58. How do you handle file uploads in Playwright, including uploads triggered by a custom (non `<input>`) button?
- 59. How do you handle and verify file downloads?
- 60. How do you handle native browser dialogs (alert, confirm, prompt)?
- 61. How do you handle a new browser tab/window opened by clicking a link (`target="_blank"`)?
- 62. How do you test geolocation-dependent features (e.g., "stores near me")?
- 63. How do you test an application's behavior across different locales/timezones/languages?
- 64. How do you simulate a slow or offline network to test loading states and error handling?
- 65. How would you test drag-and-drop functionality?
- 66. How do you test keyboard-only navigation and accessibility flows (e.g., Tab order, Enter to submit)?
- 67. How do you integrate accessibility (a11y) checks into a Playwright suite?
- 68. How would you approach component testing with Playwright (as opposed to full E2E)?

10. Configuration, TypeScript Specifics & Best Practices

69. How do you type custom fixtures properly in TypeScript so tests get full autocomplete and type safety?
70. How do you manage environment-specific configuration (dev/staging/prod URLs, credentials) in a TypeScript Playwright project?
71. What is the purpose of `testDir`, `testMatch`, and `testIgnore` in `playwright.config.ts`, and how would you use them to separate a UI suite from an API suite?
72. How do you define global setup and teardown that runs once before/after the entire test suite (not per test/worker)?
73. What is the difference between the Playwright Test runner (`@playwright/test`) and using `playwright-core/playwright` directly as a library?
74. How would you explain the key architectural differences between Playwright and Selenium that matter in real-world test reliability?
75. What are the most important best practices you'd enforce in a real Playwright + TypeScript codebase for long-term maintainability?

1. Locators, Selectors & Element Handling

Q1. Why does Playwright recommend user-facing locators (`getByRole`, `getByLabel`, `getByText`) over CSS/XPath in real projects?

User-facing locators query the accessibility tree the same way a real user or screen reader perceives the page, so they survive DOM restructuring, class-name refactors, and CSS framework changes. They also force teams to keep the UI accessible, since a missing role or label breaks the test the same way it breaks accessibility. CSS/XPath locators are tightly coupled to implementation details (div nesting, class names) that change frequently during active development, making them brittle in real sprints where the front end changes often.

```
await page.getByRole('button', { name: 'Submit order' }).click();
await page.getByLabel('Email address').fill('test@example.com');
await page.getByText('Order confirmed', { exact: true }).waitFor();
```

Q2. What is the difference between `locator.first()`, `locator.nth()`, and `locator.last()`, and when would using them be a code smell?

`first()`, `nth(index)`, and `last()` narrow a multi-element locator to a single element by position. They are useful when a list genuinely has no unique identifying text (e.g., "pick any row"). They become a code smell when used to work around a non-unique locator that should instead be scoped more precisely (e.g., by row data or a test-id), because position-based selection breaks silently when the list order changes, sort order changes, or an item is inserted — the test still passes but is asserting on the wrong row.

Q3. How do you scope a locator to search only within a specific parent element (real-world table/row scenario)?

Chain locators so the second query is evaluated only within the first's matched element(s), which is essential when the same text/button appears in many rows of a table or list.

```
const row = page.locator('tr', { hasText: 'INV-1042' });
await row.getByRole('button', { name: 'Refund' }).click();
```

Q4. What is the difference between `locator.filter()` and passing a `hasText`/`has` option directly in `locator()`?

Both ultimately filter a set of matched elements, but `filter()` is chainable and composable after the locator is already created (useful for building the locator step by step or reusing a base locator across multiple filtered variants), while `has`/`hasText` passed inline is a one-shot filter at creation time. In real projects, `filter()` with a "has" sub-locator is preferred when you need to find a row containing a specific child element (e.g., an icon or badge) rather than plain text.

```
const activeRows = page.locator('tr').filter({ has: page.getByText('Active') });
const cancelledRows = page.locator('tr').filter({ hasNotText: 'Cancelled' });
```

Q5. How do you handle a dynamic list where you need to assert the count of elements before interacting with any of them?

Use the built-in web-first assertion `expect(locator).toHaveCount()` which auto-retries until the count stabilizes, instead of reading `locator.count()` once, because a single `count()` call can race against the page still rendering items asynchronously (e.g., after an API call populates a grid).

```
await expect(page.getByRole('row')).toHaveCount(11); // header + 10 rows
const rows = await page.getByRole('row').all();
```

Q6. How do you build a robust locator strategy when developers don't add data-testid attributes and the design changes often?

Prioritize in this order for a real project: role + accessible name (`getByRole`), then label/placeholder/text (`getByLabel`, `getByPlaceholder`, `getByText`), then a stable structural anchor combined with relative navigation (`locator('section').getByRole(...)`), and only fall back to CSS as a last resort, ideally scoped rather than a long brittle selector chain. Push back with developers to add data-testid for elements with no accessible role (icon-only buttons, custom widgets) since that is cheaper long-term than maintaining fragile CSS.

Q7. How do you interact with elements inside an iframe using Playwright?

Playwright treats a frame as its own locator context via `page.frameLocator(selector)`, and all subsequent locator calls are automatically scoped and auto-waited inside that frame — there's no need to manually switch context like in Selenium.

```
const frame = page.frameLocator('#payment-iframe');
await frame.getByLabel('Card number').fill('4242424242424242');
await frame.getByRole('button', { name: 'Pay' }).click();
```

Q8. How do you interact with elements inside a Shadow DOM in Playwright?

Playwright pierces open shadow roots automatically — regular locators (CSS, `getByRole`, `getByText`) traverse into shadow DOM by default without extra syntax, unlike Selenium which requires manually querying the `shadowRoot`. Closed shadow roots cannot be pierced by any tool, including Playwright, since the browser itself hides them.

Q9. How would you handle a custom dropdown/autocomplete component that isn't a native `<select>` element?

Treat it as a sequence of real user actions rather than trying to "select" a value: click to open the dropdown, type to filter (if it's a searchable combobox), wait for the option list (often role="listbox"/"option"), then click the matching option — using web-first assertions to wait for the option to appear rather than a hard sleep.

```
await page.getByRole('combobox', { name: 'Country' }).click();
await page.getByRole('combobox', { name: 'Country' }).fill('Ireland');
await page.getByRole('option', { name: 'Ireland' }).click();
```

Q10. How do you handle native `<select>` dropdowns versus custom-rendered ones?

For a true HTML `<select>`, use `locator.selectOption()` which handles the native browser dropdown directly and doesn't need clicking to "open" it. For a custom `div/ul`-based dropdown, `selectOption` won't work at all since there's no underlying `<select>` element — you must simulate real clicks as shown in the combobox example above.

```
await page.getByLabel('Country').selectOption({ label: 'Ireland' });
await page.getByLabel('Country').selectOption('IE');
```

2. Waiting Strategies & Test Reliability

Q11. Explain Playwright's auto-waiting mechanism and which conditions it checks before an action like `click()`.

Before performing an action, Playwright automatically waits for the target element to be attached to the DOM, visible, stable (not animating), able to receive events (not obscured by another element), and enabled — all within the configured `actionTimeout`. This eliminates most explicit waits needed in Selenium, but it does NOT wait for unrelated network calls or custom app-level "loading" states outside the element's own readiness.

Q12. If auto-waiting exists, why do flaky tests still happen, and how do you fix them in real projects?

Flakiness usually comes from state that auto-waiting can't see: a button becomes clickable before its `onClick` handler has finished attaching, an API call fires after the click but the assertion checks the UI before the response updates it, or a toast/animation timing race. The fix is to assert on the actual outcome (network response, resulting UI state) rather than the action itself, and to use web-first assertions (`expect(locator).toHaveText(...)`) that retry, rather than a single equality check.

```
const [response] = await Promise.all([
  page.waitForResponse(res => res.url().includes('/api/orders') && res.status() === 200),
  page.getByRole('button', { name: 'Place order' }).click(),
]);
await expect(page.getByText('Order placed')).toBeVisible();
```

Q13. What is the difference between `page.waitForSelector()` and `expect(locator).toBeVisible()`?

`waitForSelector()` is a legacy, imperative wait that resolves once and returns a handle; it's still available but discouraged in modern Playwright Test. `expect(locator).toBeVisible()` is a web-first assertion — it polls automatically, integrates with the test reporter (giving a clear diff on failure), and is the idiomatic way to wait for and assert visibility in one step within `@playwright/test`.

Q14. How does `expect.poll()` differ from a normal web-first assertion, and when would you use it in a real scenario?

Web-first assertions like `toHaveText()` only work on Playwright locators/responses. `expect.poll()` lets you retry-assert on any arbitrary async function — for example, polling a database row, a queue length, or a third-party REST API until a condition is met — which is common in real end-to-end tests that verify backend side effects of a UI action (e.g., "an email was queued after checkout").

```
await expect.poll(async () => {
  const res = await fetch(`${apiBase}/orders/${orderId}`);
  return (await res.json()).status;
}, { timeout: 15000 }).toBe('CONFIRMED');
```

Q15. How do you avoid hard-coded waits (`page.waitForTimeout`) in a CI pipeline where tests need to be reliable and fast?

Replace fixed sleeps with condition-based waits: `waitForResponse`/`waitForRequest` for network calls, `toBeVisible`/`toHaveText` for UI state, `waitForLoadState('networkidle')` sparingly (it's discouraged for SPAs with polling/websockets), or `waitForFunction` for custom JS conditions. Hard waits either fail intermittently (too short) or slow the whole suite down needlessly (too long) — condition-based waits are both faster and more reliable.

Q16. How do you handle a page where content loads via multiple sequential API calls (e.g., a dashboard that fires several XHRs on load)?

Rather than waiting for a specific number of network calls, assert on the final rendered state the user actually cares about (e.g., `toHaveText`, `toBeVisible` on the last-populated widget), since that inherently waits for every upstream dependency to resolve. If you must synchronize on a specific call (e.g., to validate the payload), use `waitForResponse` with a URL/predicate match for that specific request rather than `networkidle`, which is unreliable for pages with polling or analytics beacons.

Q17. What causes a "strict mode violation" error in Playwright and how do you resolve it?

Locators in Playwright Test operate in strict mode by default — any action or assertion on a locator that resolves to more than one DOM element throws an error instead of silently picking the first one, to prevent tests from silently interacting with the wrong element. It's resolved by scoping the locator further (adding `hasText`, `filter`, or chaining into a parent), or explicitly picking an element with `.first()/.nth()` when that's genuinely intended.

Q18. How do you handle elements that are visible in the DOM but covered by another element (e.g., a sticky header or a cookie banner)?

Playwright's actionability checks already fail fast with a clear "element is outside of the viewport" or "intercepts pointer events" error rather than silently mis-clicking, which is the correct real-world signal to dismiss the blocking element first (e.g., accept the cookie banner) rather than forcing the click. Use `{ force: true }` only as a last resort, understanding it skips actionability checks entirely and can mask a real UX bug.

3. Test Architecture, Fixtures & Page Object Model

Q19. How do you implement the Page Object Model (POM) in Playwright using TypeScript in a maintainable way?

Define one class per page/component with locators as readonly properties initialized in the constructor and methods representing user workflows (not raw actions), so tests read like business steps. Locators are lazily created (calling `page.locator()` doesn't query the DOM immediately), so defining them in the constructor is cheap and keeps them centralized.

```
export class LoginPage {
  readonly emailInput = this.page.getByLabel('Email');
  readonly passwordInput = this.page.getByLabel('Password');
  readonly signInButton = this.page.getByRole('button', { name: 'Sign in' });

  constructor(private page: Page) {}

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.signInButton.click();
  }
}
```

Q20. How do custom fixtures in Playwright Test improve on a traditional POM-only setup?

Fixtures provide automatic setup/teardown, dependency injection, and lifecycle scoping (test vs worker), so page objects, authenticated pages, or test data are created only when a test actually requests them and cleaned up automatically. This avoids repetitive `beforeEach` boilerplate and lets each test declare exactly what it needs.

```
type MyFixtures = { loginPage: LoginPage; dashboardPage: DashboardPage };

export const test = base.extend<MyFixtures>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
  dashboardPage: async ({ page }, use) => {
    await use(new DashboardPage(page));
  },
});
```

Q21. What is the difference between test-scoped and worker-scoped fixtures, and give a real use case for worker scope?

Test-scoped fixtures (the default) are created fresh for every single test and torn down after it. Worker-scoped fixtures ({ scope: 'worker' }) are created once per worker process and reused across every test that runs in that worker, which is ideal for expensive, reusable resources like a database connection pool, a shared test-data seeding client, or spinning up a mock server once instead of per test.

```
export const test = base.extend<{}, { apiClient: ApiClient }>({
  apiClient: [async ({}, use) => {
    const client = await ApiClient.connect();
    await use(client);
    await client.close();
  }, { scope: 'worker' }],
});
```

Q22. How do you merge fixtures from multiple files/plugins (e.g., combining your own POM fixtures with an auth fixture)?

Use mergeTests (or repeated .extend() calls) to combine independently-defined fixture sets into a single test object that tests import, keeping each fixture file focused on one concern (auth, page objects, test data) instead of one giant fixtures file.

```
import { mergeTests } from '@playwright/test';
import { test as authTest } from './auth.fixtures';
import { test as pomTest } from './pom.fixtures';

export const test = mergeTests(authTest, pomTest);
```

Q23. How do you structure a large Playwright project (100+ specs) so it stays maintainable across multiple teams?

Separate concerns into folders: /tests (specs only), /pages (POMs), /fixtures (custom fixtures), /utils (helpers, API clients), /test-data (fixtures/builders for data), and /config (multiple playwright.config.ts per environment if needed). Use projects in the config to isolate suites (smoke, regression, api) so teams can run only their slice in CI, and enforce a shared ESLint/TS config so page objects and locator conventions stay consistent across teams.

Q24. What is the "test.step" API and why is it valuable for real-world debugging and reporting?

test.step() wraps a logical chunk of a test (e.g., "log in", "add item to cart") so it shows as a collapsible, timed node in the HTML report and trace viewer, making it far easier to see exactly which business step failed in a long end-to-end flow rather than staring at a wall of individual actions.

```
await test.step('Add product to cart', async () => {
  await page.getByRole('button', { name: 'Add to cart' }).click();
  await expect(page.getByText('1 item in cart')).toBeVisible();
});
```

Q25. How do you avoid test interdependence (tests relying on order/state from a previous test) in Playwright?

Each test should get a fresh, isolated browser context by default (Playwright Test does this automatically per test), so no cookies/localStorage/session leak between tests. On top of that, seed and clean up test data via API calls in fixtures rather than relying on UI state left over from another test, and never rely on tests running in a specific order — configure `fullyParallel: true` so ordering assumptions surface as failures early.

4. API Testing, Network Interception & Mocking

Q26. How do you make a pure API request (no browser) using Playwright, and why would you do that in an E2E suite?

Playwright ships an `APIRequestContext` (request fixture) that can make HTTP calls directly, which is useful for fast test-data setup/teardown (e.g., creating a user via API instead of the UI) and for hybrid tests that validate a UI action's backend effect without a slow round-trip through browser rendering.

```
test('create order via API then verify in UI', async ({ request, page }) => {
  const res = await request.post('/api/orders', { data: { sku: 'ABC123', qty: 2 } });
  expect(res.ok()).toBeTruthy();
  const order = await res.json();
  await page.goto(`/orders/${order.id}`);
  await expect(page.getByText('ABC123')).toBeVisible();
});
```

Q27. How do you intercept and mock a network response to test a UI's error-handling state (e.g., a 500 error banner)?

Use `page.route()` to intercept requests matching a URL pattern and call `route.fulfill()` with a custom status/body instead of letting the real request through, which lets you deterministically test edge-case UI states (errors, empty states, slow responses) that are hard to reproduce against a real backend.

```
await page.route('**/api/orders', route =>
  route.fulfill({ status: 500, body: JSON.stringify({ error: 'Internal error' }) })
);
await page.goto('/orders');
await expect(page.getByText('Something went wrong')).toBeVisible();
```

Q28. What is the difference between `route.fulfill()`, `route.continue()`, and `route.abort()`?

`fulfill()` short-circuits the request and returns a custom response without hitting the network at all (used for mocking). `continue()` lets the request proceed, optionally with modified headers/method/postData (used for tampering with a real request, e.g., injecting a header). `abort()` cancels the request entirely with a given error reason, useful for simulating offline mode or a failed third-party script (like an analytics call) without breaking the rest of the page.

Q29. How would you test a page that depends on a slow third-party API without making your suite slow or flaky?

Intercept and mock the third-party call so the test controls its timing and payload deterministically, rather than depending on a real external service's latency or availability, which also removes a source of flakiness unrelated to the app under test.

```
await page.route('**/thirdparty.com/**', async route => {
  await route.fulfill({ status: 200, body: JSON.stringify({ rate: 1.08 }) });
});
```

Q30. How do you validate the actual request payload sent by the frontend (e.g., asserting a POST body is correct) rather than just the response?

Capture the request via `page.waitForRequest()` or by inspecting it inside a route handler before calling `continue()/fulfill()`, then assert on `request.postDataJSON()` or headers — this verifies the frontend is sending correct data, which pure UI assertions can't catch.

```
const requestPromise = page.waitForRequest(req => req.url().includes('/api/checkout') && req.method() === 'POST');
await page.getByRole('button', { name: 'Checkout' }).click();
const req = await requestPromise;
expect(req.postDataJSON()).toMatchObject({ currency: 'EUR' });
```

Q31. How do you test a page that uses WebSockets, and can Playwright intercept WebSocket traffic?

Playwright can listen to WebSocket frames via `page.on('websocket', ws => ...)` and its `framesent/framereceived` events for observability/debugging, but unlike HTTP it cannot mock/fulfill WebSocket messages the way `route.fulfill()` does for HTTP — so for deterministic WS testing you typically run against a real or a test-double WebSocket server, or assert on the resulting UI state driven by the socket messages rather than the wire protocol itself.

Q32. How do you globally mock all API calls for an entire test file or test run (not just one test)?

Set up the route interception in a `beforeEach` hook (or a fixture) so it's registered before `page.goto()` runs for every test in the file/describe block, ensuring the mock is active from the very first navigation instead of racing the app's initial data-fetch on load.

```
test.beforeEach(async ({ page }) => {
  await page.route('**/api/**', route => route.fulfill({ json: mockApiResponse(route.request().url()) });
});
```

5. Authentication, Sessions & Cross-Cutting Setup

Q33. How do you avoid logging in through the UI in every single test, which is slow and adds unrelated flakiness?

Log in once in a setup project/step, save the authenticated browser state (cookies + localStorage) to a JSON file via `context.storageState()`, and reuse that file as the `storageState` for every other test's browser context — so tests start already authenticated.

```
// auth.setup.ts
setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@test.com');
  await page.getByLabel('Password').fill('secret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.context().storageState({ path: 'playwright/.auth/user.json' });
});
```

Q34. How do you configure `playwright.config.ts` to run an auth "setup" project before the actual test projects, and reuse its output?

Declare a dedicated project with `testMatch` for the setup file, then make other projects depend on it via dependencies, and point their `use.storageState` at the saved auth file — Playwright Test guarantees the setup project runs to completion first.

```
projects: [
  { name: 'setup', testMatch: /auth\.setup\.ts/ },
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'], storageState: 'playwright/.auth/user.json' },
    dependencies: ['setup'],
  },
],
```

Q35. How do you handle multiple user roles (admin, standard user, read-only) that each need separate authenticated sessions?

Create one `storageState` JSON per role by running the login setup once per role (parametrized setup tests or separate setup files), then define a project per role (or a fixture that picks the right storage file), so tests explicitly request the role fixture they need (e.g., `adminPage`, `viewerPage`) instead of one shared session.

Q36. How do you handle authentication for an app protected by SSO/OAuth (e.g., Google or Okta login) in automated tests?

Avoid automating the third-party SSO provider's real login UI in every test run, since it's slow, may trigger CAPTCHA/bot detection, and is outside the team's control. Common real-world approaches: perform the SSO login once and reuse `storageState` as above, use a test-specific identity provider or a stubbed OAuth flow provided by the backend team for test environments, or obtain a valid session/JWT via an API call and inject it directly into the browser context via `addCookies()` or an init script.

Q37. How do you handle a token that expires mid-suite (e.g., a JWT with a short TTL) so long test runs don't start failing due to auth expiry?

Either extend the token TTL specifically for the test environment, refresh it programmatically in a worker-scoped fixture with a check-and-refresh routine before each test/project run, or intercept refresh-token calls so the app's own refresh mechanism keeps the session alive — the key is not hard-coding a single static token that's expected to survive an entire CI run.

Q38. How do you seed test data (e.g., a specific user, a specific product) before a test and clean it up afterward, in a way that keeps tests independent?

Use a fixture that creates the data via a direct API/DB call before the test body runs (yielded to the test via `use()`), and deletes/rolls it back in the same fixture after `use()` resolves — this guarantees cleanup even if the test fails, since fixture teardown still runs.

```
export const test = base.extend<{ testUser: User }>({
  testUser: async ({ request }, use) => {
    const res = await request.post('/api/test-users', { data: { role: 'admin' } });
    const user = await res.json();
    await use(user);
    await request.delete(`/api/test-users/${user.id}`);
  },
});
```

6. Parallelization, CI/CD & Execution Strategy

Q39. How does Playwright Test achieve parallelism, and what's the difference between "workers" and "projects"?

Workers are separate OS processes that run test files in parallel (configured via workers in the config or --workers CLI flag); each worker loads its own browser instance(s) and runs a subset of test files, giving true process-level isolation. Projects are named configurations (e.g., chromium, firefox, mobile-safari, or api vs e2e) that can each have their own settings and run independently — the two combine, since each project's tests are distributed across the available workers.

Q40. What is the difference between fullyParallel: true and the default parallel mode?

By default, Playwright Test parallelizes across files but runs tests within the same file serially (one after another, in the same worker). Setting fullyParallel: true makes every individual test — even within the same file — eligible to run in its own worker concurrently, which maximizes speed but requires the tests to be fully independent (no shared mutable state or ordering assumptions) since siblings can genuinely run at the same time.

Q41. How do you implement test sharding across multiple CI machines, and how is it different from workers?

Workers parallelize within a single machine/process pool. Sharding (--shard=1/4, --shard=2/4, etc.) splits the entire test suite across multiple independent CI machines/jobs, each running a fraction of the files with its own set of workers — used when a suite is too large for parallelism on one machine to keep CI fast, e.g., in a GitHub Actions matrix.

```
// GitHub Actions matrix: shard: [1, 2, 3, 4]
// npx playwright test --shard=${{ matrix.shard }}/4
```

Q42. How do you merge HTML/JSON reports produced by multiple shards into a single report?

Each shard is run with the blob reporter to produce a report fragment, then a separate job downloads all fragments and runs npx playwright merge-reports to combine them into one unified HTML report — this is the standard pattern for sharded CI pipelines.

```
// per shard: npx playwright test --shard=2/4 --reporter=blob
// merge job: npx playwright merge-reports --reporter=html ./all-blob-reports
```

Q43. What retry strategy would you configure for CI vs local development, and why should retries differ?

Locally, retries are usually 0 so a developer sees a real failure immediately without waiting for retries to exhaust. In CI, a small retry count (e.g., 1–2) is common to absorb environment-level flakiness (shared infra latency, resource contention) without masking a genuinely broken feature — but retries should never be used to paper over a consistently flaky test; those should be flagged and fixed, since retries hide the true flake rate from the team.

```
retries: process.env.CI ? 2 : 0,
```

Q44. How do you set up Playwright to run inside Docker for CI, and what's a common pitfall?

Use Microsoft's official `mcr.microsoft.com/playwright` image (matching the exact Playwright npm version) which bundles all browser binaries and OS dependencies pre-installed, avoiding the common pitfall of browsers failing to launch due to missing system libraries on a bare Linux CI image. A version mismatch between the npm `@playwright/test` package and the Docker image tag is the most frequent real-world issue — they must be pinned to the same version.

Q45. How do you run only a subset of tests (e.g., "smoke" tests) in a fast pre-merge CI check, while running the full suite nightly?

Tag tests using Playwright's annotation syntax in the title or `test.info().annotations`, then filter with `--grep/--grep-invert` at the CLI, or define separate projects with a `testMatch/grep` filter so CI pipelines can trigger different `npx playwright` test invocations for pre-merge vs nightly runs.

```
test('checkout completes successfully @smoke', async ({ page }) => { /* ... */ });  
// CLI: npx playwright test --grep @smoke
```

7. Debugging, Trace Viewer & Observability

Q46. What is the Playwright Trace Viewer and what real debugging value does it add over screenshots/logs alone?

Trace Viewer records a full timeline of a test run — DOM snapshots before/after every action, network requests, console logs, and source code — letting you scrub through the test like a video and inspect the exact DOM state at the moment of failure, which is far richer than a single failure screenshot for diagnosing why a locator didn't match or why an assertion failed.

```
use: { trace: 'on-first-retry' },  
// CLI to view: npx playwright show-trace trace.zip
```

Q47. What's the tradeoff of setting trace: 'on' for every run versus 'on-first-retry' or 'retain-on-failure'?

trace: 'on' captures a trace for every single test (including passing ones), which gives maximum forensic detail but produces large artifacts and slows the run/increases storage costs at scale. 'on-first-retry' only records a trace when a test fails and is retried, and 'retain-on-failure' keeps the trace only for failed tests — both give you debugging data exactly when you need it (a failure) without the overhead of tracing hundreds of passing tests.

Q48. How do you debug a single failing test locally with step-by-step control (not just headless output)?

Use the Playwright Inspector via PWDEBUG=1 or the --debug CLI flag, which opens the browser alongside a UI that lets you step through each action, inspect locators live, and use the "pick locator" tool — this is far more effective than reading console output for figuring out why a locator isn't matching.

```
// npx playwright test login.spec.ts --debug  
// or: PWDEBUG=1 npx playwright test
```

Q49. What is Playwright's UI Mode and how does it help in day-to-day development?

UI Mode (npx playwright test --ui) is an interactive, watch-mode test runner with a timeline, DOM snapshots per action, and a pick-locator tool, letting a developer write and iteratively re-run a single test while watching exactly what the browser did at each step — it's typically faster for day-to-day authoring than repeatedly running headless and reading a report.

Q50. How do you capture and attach custom debugging artifacts (e.g., a database dump or a specific API response) to the Playwright HTML report for a failing test?

Use testInfo.attach() inside the test to add arbitrary files/buffers to the report alongside the built-in screenshots/traces, which is valuable when the root cause of a failure often lives outside the browser (e.g., attaching the raw API response body that caused an unexpected UI state).

```
test('checkout', async ({ page }, testInfo) => {  
  const res = await page.request.get('/api/cart');  
  await testInfo.attach('cart-response', { body: await res.text(), contentType: 'application/json' });  
});
```

Q51. A test passes locally but consistently fails in CI. What is your systematic approach to diagnosing this?

Check environment differences first: viewport/OS/browser version defaults, headless vs headed rendering differences (fonts, animations), timezone/locale, network latency to a shared test environment, and resource contention from parallel workers. Then pull the CI trace/video/screenshot artifacts (which should always be configured to upload on failure) to see the actual failure state, and reproduce locally using the same headless mode, viewport, and worker count as CI (`--workers=1 --headed=false`) before assuming it's "just flaky."

8. Assertions, Visual Testing & Cross-Browser

Q52. What is a "web-first assertion" in Playwright and how does it differ from a plain Node.js assert/expect equality check?

Web-first assertions (`expect(locator).toBeVisible()`, `toHaveText()`, `toHaveValue()`, etc.) automatically retry against the live page until the condition is true or the timeout elapses, whereas a plain equality check (`expect(await locator.textContent()).toBe('x')`) reads the DOM exactly once — meaning it fails if the text hasn't updated yet even a millisecond after the check, a very common source of flakiness when developers use the wrong pattern.

Q53. What is a "soft assertion" and when is it appropriate to use in a real test?

`expect.soft()` records a failure but lets the test continue executing instead of throwing immediately, which is useful when you want to check multiple independent things in one test run (e.g., verifying five different fields on a summary page) and see all failures in one report instead of stopping at the first one. It should be used sparingly — for truly independent checks — not as a substitute for splitting a test into smaller, focused tests.

```
await expect.soft(page.getByTestId('total')).toHaveText('€49.99');
await expect.soft(page.getByTestId('tax')).toHaveText('€9.50');
expect(test.info().errors).toHaveLength(0); // fails the test at the end if any soft assertion failed
```

Q54. How do you implement visual regression testing with Playwright, and what's the biggest real-world challenge with it?

`expect(page).toHaveScreenshot()` (or `locator-scoped`) compares a rendered screenshot against a stored baseline, failing if the pixel diff exceeds a threshold. The biggest real-world challenge is baseline instability across OS/browser/font-rendering differences and dynamic content (timestamps, ads, animations) — teams mitigate this by running visual tests in a fixed Docker environment matching CI exactly, masking dynamic regions, and disabling animations.

```
await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.getByTestId('last-updated-timestamp')],
  maxDiffPixelRatio: 0.02,
});
```

Q55. How do you handle animations/transitions that cause visual tests to be flaky?

Set `animations: 'disabled'` on the `toHaveScreenshot()/toHaveScreenshot()` call (or globally in config), which makes Playwright automatically finish/skip CSS transitions and animations before capturing, ensuring the screenshot always reflects the final settled state rather than an arbitrary mid-animation frame.

Q56. How do you run the same test suite across Chromium, Firefox, and WebKit, and what real cross-browser issues commonly surface?

Define multiple projects in `playwright.config.ts`, one per browser/device via devices presets, and Playwright Test runs the full suite against each automatically. Common real issues: WebKit's stricter handling of file inputs/downloads and slightly different default fonts affecting visual tests, Firefox's different handling of focus/blur timing on form fields, and Safari-specific date/input picker rendering differences.

```
projects: [  
  { name: 'chromium', use: { ...devices['Desktop Chrome'] } },  
  { name: 'firefox', use: { ...devices['Desktop Firefox'] } },  
  { name: 'webkit', use: { ...devices['Desktop Safari'] } },  
],
```

Q57. How do you test responsive/mobile layouts without a real device farm?

Use Playwright's built-in device descriptors (`devices['iPhone 13']`, `devices['Pixel 5']`) which set viewport, user agent, touch support, and device scale factor to emulate real devices, combined with projects to run key flows against multiple device profiles — sufficient for layout/behavior testing, though real device farms (BrowserStack, Sauce Labs) are still used for OS-specific rendering/hardware quirks.

9. Advanced Real-World Scenarios

Q58. How do you handle file uploads in Playwright, including uploads triggered by a custom (non `<input>`) button?

For a real `<input type="file">`, use `locator.setInputFiles()` directly, even if the input is visually hidden, since Playwright can interact with hidden file inputs. For a custom button that programmatically opens the OS file picker, listen for the `'filechooser'` event and resolve it, since the native OS dialog itself can't be automated.

```
await page.getByLabel('Upload resume').setInputFiles('fixtures/resume.pdf');

const [chooser] = await Promise.all([
  page.waitForEvent('filechooser'),
  page.getByRole('button', { name: 'Upload file' }).click(),
]);
await chooser.setFiles('fixtures/resume.pdf');
```

Q59. How do you handle and verify file downloads?

Listen for the `'download'` event around the triggering action, then use `download.path()/download.saveAs()` to access the file and assert on its name, size, or contents (e.g., parsing a downloaded CSV) — Playwright automatically manages the download without needing OS-level file dialog automation.

```
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export CSV' }).click(),
]);
expect(download.suggestedFilename()).toBe('report.csv');
const path = await download.path();
```

Q60. How do you handle native browser dialogs (alert, confirm, prompt)?

Register a `'dialog'` event listener before the action that triggers the dialog, since native dialogs block the page and must be explicitly accepted or dismissed programmatically — Playwright auto-dismisses unhandled dialogs by default, which can silently break a flow if you don't register a handler for an expected `confirm()`.

```
page.once('dialog', dialog => dialog.accept());
await page.getByRole('button', { name: 'Delete account' }).click();
```

Q61. How do you handle a new browser tab/window opened by clicking a link (`target="_blank"`)?

Listen for the `'popup'` event (or `context.waitForEvent('page')`) around the click, which gives you a handle to the new Page object so you can continue interacting with or asserting on the newly opened tab independently of the original page.

```
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.getByRole('link', { name: 'Open in new tab' }).click(),
]);
await newPage.waitForLoadState();
await expect(newPage).toHaveURL(/checkout/);
```

Q62. How do you test geolocation-dependent features (e.g., "stores near me")?

Grant the geolocation permission and set coordinates via context options (or `context.setGeolocation()` at runtime), which lets you deterministically test location-based behavior without a real GPS/device.

```
const context = await browser.newContext({
  geolocation: { latitude: 53.3498, longitude: -6.2603 },
  permissions: ['geolocation'],
});
```

Q63. How do you test an application's behavior across different locales/timezones/languages?

Set locale and timezoneId in the browser context options (and optionally extraHTTPHeaders for Accept-Language if the backend drives localization from headers), letting you assert on date formatting, currency formatting, and translated strings per test without needing separate physical environments.

```
const context = await browser.newContext({ locale: 'de-DE', timezoneId: 'Europe/Berlin' });
```

Q64. How do you simulate a slow or offline network to test loading states and error handling?

Use `context.setOffline(true)` to simulate a fully offline network, or route interception with an artificial delay (`await new Promise(r => setTimeout(r, ms))` inside a route handler before calling `continue()/fulfill()` to simulate slow responses, letting you deterministically assert on spinners, skeleton loaders, or offline banners.

```
await page.route('**/api/**', async route => {
  await new Promise(r => setTimeout(r, 3000));
  await route.continue();
});
await expect(page.getByTestId('loading-spinner')).toBeVisible();
```

Q65. How would you test drag-and-drop functionality?

Playwright supports it via `locator.dragTo()` for simple cases (source and target elements with standard HTML5 drag events), or manual mouse choreography (`hover`, `mouse.down()`, `mouse.move()` in steps, `mouse.up()`) for custom drag implementations (e.g., canvas-based or libraries with custom event handling) where `dragTo`'s built-in event simulation isn't sufficient.

```
await page.getByText('Task A').dragTo(page.getByTestId('in-progress-column'));
// manual fallback:
await source.hover();
await page.mouse.down();
await target.hover();
await page.mouse.up();
```

Q66. How do you test keyboard-only navigation and accessibility flows (e.g., Tab order, Enter to submit)?

Use `page.keyboard.press()` to simulate real key events (Tab, Enter, Escape, Shift+Tab) and assert focus state via `toBeFocused()` on the expected element, which validates both keyboard operability and correct tab order — an increasingly common requirement in real projects with accessibility (WCAG) compliance obligations.

```
await page.keyboard.press('Tab');
await expect(page.getByLabel('Email')).toBeFocused();
await page.keyboard.press('Enter');
```

Q67. How do you integrate accessibility (a11y) checks into a Playwright suite?

Use the `@axe-core/playwright` package to run automated WCAG audits against a page/component within a test and assert there are zero critical violations, which catches issues like missing alt text or insufficient color contrast automatically as part of regular E2E runs, though it supplements rather than replaces manual accessibility review.

```
import AxeBuilder from '@axe-core/playwright';
const results = await new AxeBuilder({ page }).analyze();
expect(results.violations).toEqual([]);
```

Q68. How would you approach component testing with Playwright (as opposed to full E2E)?

Playwright's experimental Component Testing (`@playwright/experimental-ct-react/vue/svelte`) mounts a single component in a real browser in isolation via `mount()`, letting you test component-level behavior/props/events with real browser rendering and Playwright's locator/assertion API, faster than a full E2E flow and more realistic than a jsdom unit test, useful for design-system or component-library projects.

10. Configuration, TypeScript Specifics & Best Practices

Q69. How do you type custom fixtures properly in TypeScript so tests get full autocomplete and type safety?

Pass explicit generic type parameters to `test.extend<TestFixtures, WorkerFixtures>({...})`, declaring an interface/type for the fixtures object — TypeScript then infers the correct fixture types inside every test callback that destructures them, giving compile-time errors if a fixture name is misspelled or misused.

```
type TestFixtures = { loginPage: LoginPage };
type WorkerFixtures = { apiClient: ApiClient };

export const test = base.extend<TestFixtures, WorkerFixtures>({
  loginPage: async ({ page }, use) => use(new LoginPage(page)),
  apiClient: [async ({}, use) => use(new ApiClient()), { scope: 'worker' }],
});
```

Q70. How do you manage environment-specific configuration (dev/staging/prod URLs, credentials) in a TypeScript Playwright project?

Use `dotenv` to load environment-specific `.env` files (`.env.staging`, `.env.prod`) selected via a CI variable, referenced in `playwright.config.ts`'s `use.baseUrl` and `process.env`, combined with TypeScript's `declare global` augmentation of `ProcessEnv` for typed env access — secrets themselves come from CI secret stores, never committed to the repo.

```
// playwright.config.ts
import dotenv from 'dotenv';
dotenv.config({ path: `./.env.${process.env.TEST_ENV ?? 'dev'}` });

export default defineConfig({
  use: { baseUrl: process.env.BASE_URL },
});
```

Q71. What is the purpose of `testDir`, `testMatch`, and `testIgnore` in `playwright.config.ts`, and how would you use them to separate a UI suite from an API suite?

`testDir` sets the root folder Playwright scans for spec files; `testMatch`/`testIgnore` are glob/regex filters that include or exclude specific files within it. In practice, teams often keep `/tests/ui` and `/tests/api` in the same repo but define separate projects, each with its own `testMatch` pattern (or `testDir`), so `...playwright test --project=api` can run only API specs without launching a browser at all.

Q72. How do you define global setup and teardown that runs once before/after the entire test suite (not per test/worker)?

Point `globalSetup`/`globalTeardown` in `playwright.config.ts` to files exporting a default async function — these run once for the whole run (e.g., seeding a shared database, starting a mock server, or authenticating a service account) before any worker starts, and after all workers finish, respectively.

```
export default defineConfig({
  globalSetup: require.resolve('./global-setup'),
  globalTeardown: require.resolve('./global-teardown'),
});
```

Q73. What is the difference between the Playwright Test runner (@playwright/test) and using playwright-core/playwright directly as a library?

@playwright/test is a full-featured test runner built on top of the core library, adding fixtures, parallelization, retries, reporters, assertions, and config — the standard choice for writing test suites. The raw playwright (or playwright-core) package is just the browser automation API with no test runner, used when embedding browser automation into a non-test context (a scraping script, a custom framework, or a Node.js backend service) rather than an actual test suite.

Q74. How would you explain the key architectural differences between Playwright and Selenium that matter in real-world test reliability?

Playwright communicates with browsers over a single persistent WebSocket-based protocol (CDP for Chromium, custom protocols for Firefox/WebKit) controlled directly by the library, whereas Selenium goes through the WebDriver protocol (HTTP requests to a driver process per browser), which is generally slower and requires managing separate driver binaries. Playwright's built-in auto-waiting/actionability checks, native network interception, multiple browser contexts per browser instance (fast, isolated "incognito" sessions without spinning up new browser processes), and first-class TypeScript support are the practical differentiators that reduce flakiness and boilerplate compared to Selenium.

Q75. What are the most important best practices you'd enforce in a real Playwright + TypeScript codebase for long-term maintainability?

Use user-facing locators and avoid brittle CSS/XPath; keep tests independent and idempotent with fresh state per test via fixtures, not shared global state; assert on outcomes (network responses, resulting UI state) instead of implementation details; never use hard-coded `waitForTimeout`; centralize locators/workflows in Page Objects or fixtures rather than duplicating them per spec; enable tracing/screenshots/video only on failure to keep CI artifacts manageable; tag and separate smoke vs full regression suites; enforce strict TypeScript (no any on fixtures/POMs) so refactors are caught at compile time; and treat flaky tests as bugs to fix immediately, not something to silence with retries.